

ELTE EÖTVÖS LORÁND UNIVERSITY

AUTHENTICATION AND AUTHORIZATION

Modern web application development in .NET

Lecture 6

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

AUTHENTICATION

User management on a website or in a webservice has various elements that need to be implemented securely:

- *registration, data modification*
- *login, logout, automatic login*
- *forgotten password/username management*
  - as the password is always stored in an encrypted form, in case of a forgotten password, the user must be provided with a new password (if we are sure of its identity)
- *extra functionalities*: user groups, data visibility management, email confirmation, etc.

ASP.NET CORE IDENTITY

Due to the complexity of user authentication, ASP.NET Core provides us its own framework for it, named *ASP.NET Core Identity*

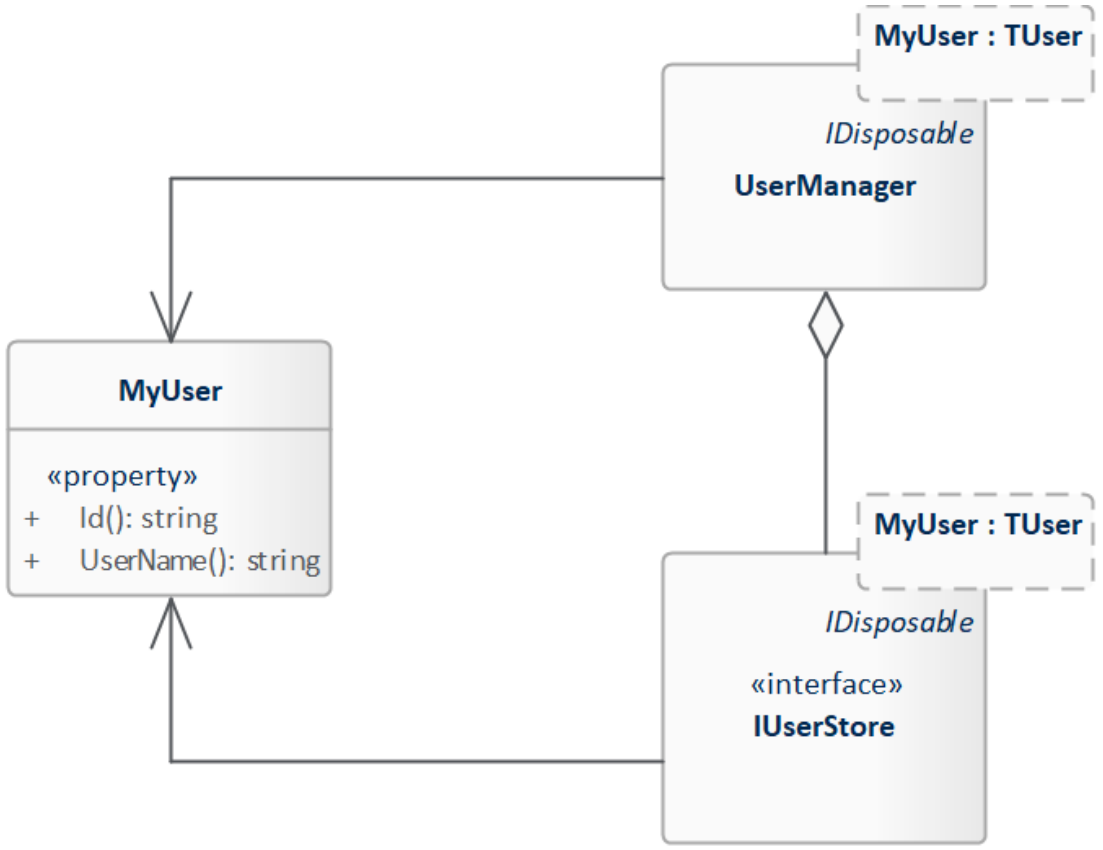
- provides the required functionality for user management, from data manegement to user interface
- provides multiple options for storing/accessing user data
  - we can use internal solutions (e.g. custom database, *Windows Authentication*), or external services (e.g. *Microsoft*, *Google*, *X (Twitter)*, *Facebook*), *OAuth* and *OpenId* are supported in general.
- The basic functionality is available from the **`Microsoft.AspNetCore.Identity`** library (*NuGet* package), with data storage provided by additional packages.

USER MANAGEMENT

User management is done by the generic **`UserManager<T>`** class, which can handle any specific user type and its operations support asynchronous execution

- the user can be registered (**`CreateAsync`**), identified (**`FindByIdAsync`**, **`FindByEmailAsync`**, etc.), modified (**`ChangePasswordAsync`**), etc.
- the user can be of any type, represented by the required information (password, email address, etc.)
- the manager class decouples how to store users, it stores the users through a repository (**`IUserStore<T>`**)
    - the user store defines the specific data management solution
- each class is specialized for the type of the user

USER MANAGEMENT



USER MANAGEMENT

- e.g.:

```
public class MyUser { … } // type of the user
public class MyStore : IUserStore<MyUser> { … }
// type of the user store, specialized to the user type

UserManager<MyUser> manager =
  new UserManager<MyUser>(new MyStore(), …);
  // user manager, specialized to the user type

MyUser user = await manager.FindByNameAsync(name);
// find user (by name)
```

USER STORAGE IN EF

The easiest way to store user data is to manage it through a local database with an ORM approach, using *Entity Framework* (add the **`Microsoft.AspNetCore.Identity.EntityFrameworkCore`** package)

- the entity model (**`IdentityDbContext`**) can handle users, identified by a username and password (**`IdentityUser`**)
  - the **`IdentityUser`** can be specialized through inheriting from it, any property can be added to the user
  - the database scheme is generated based on the data provided in the code (*code first* approach)
  - provides password encryption (with time-dependent salting)
- the entity model can be used with the provided **`UserStore`** class.

USER STORAGE IN EF

- e.g.:

```
IndentityDbContext<IdentityUser> context = new …;
// database entity model
UserStore<IdentityUser> store =
  new UserStore<IdentityUser>(context);
  // user data store
UserManager<IdentityUser> userManager =
  new UserManager<IdentiyUser>(store);
  // user manager

IdentityUser user = await userManager
                          .FindByNameAsync(name);
                        // find user
```

IDENTIFYING USERS

User login and identity is managed by the **`SignInManager<T>`** class

- is also specialized for the user type as a generic parameter, and provides asynchronous operations;
- direct login based on username-password pairs is handled by **`PasswordSignInAsync()`**, and logout by **`SignOutAsync()`** operations
  - can be specified to persist the login (i.e. remember the data) and lockout the user in case of incorrect data
- supports authentication via external authentication providers (**`ExternalLoginSignInAsync()`**) and two-factor authentication (**`TwoFactorAuthenticatorSignInAsync()`**)

IDENTIFYING USERS

- e.g. (Login):

```
public async Task<IActionResult> Login(…) {
  // ...
  var result = await signInManager.PasswordSignInAsync(
                          userName, password, remember, false);
  // login
  if(result.Succeeded) { … }
}
```

- e.g. (Logout):

```
public async Task<IActionResult> Logout() {
  await signInManager.SignOutAsync();
}
```

CONFIGURING IDENTITY

ASP.NET Core Identity has to be properly configured for the application

- the *identity type* (user type) can be registered to *IoC container* through the **`builder.Services`** service collection (**`IServiceProvider`**) in the **`Program`** class using **`AddIdentity<MyUser>()`**
  - Identity rules (e.g. required password strength) can also be configured as part of this
- the user store type can be registered with **`AddUserStore<MyStore>()`**
- when using EF, **`AddEntityFrameworkStores<MyDbContext>()`** sets up an Entity Framework Core based data store and specifies the database entity context type to be used

CONFIGURING IDENTITY

- e.g.:

```
builder.Services.AddIdentity<MyUser>(options => {
  // settings
  options.Password.RequireDigit = true;
  options.Password.RequiredLength = 8;
  options.Password.RequireNonAlphanumeric = false;
  options.Password.RequireUppercase = true;
  options.Password.RequireLowercase = false;
  options.Password.RequiredUniqueChars = 3;
  options.User.RequireUniqueEmail = true;
})
  .AddEntityFrameworkStores<MyDbContext>()
  .AddDefaultTokenProviders();
```

CONFIGURING IDENTITY

- The configured user management also must be enabled in the **`Program`** class on the **`WebApplication`** object:
  `app.UseAuthentication();`
- It is imperative to call the following statements in the **`Program`** class exactly in this order:

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
```

AUTHENTICATION

The logged in user can also be managed via the **`HttpContext`** class

- the **`User.Identity.IsAuthenticated`** property indicates whether a user has logged in
- the username of the logged in user can be accessed through the **`User.Identity.Name`** property
- e.g.:

```
if (User.Identity.IsAuthenticated) {
  IdentityUser user = await userManager.
    FindByNameAsync(User.Identity.Name);
    // retrieve the logged in user
}
```

ACCESSING REQUEST AND RESPONSE CONTEXT

- The **`HttpContext`** property is available in controllers (descendants of the **`ControllerBase`** class).
- In other classes, e.g. a business logic service, it can be accessed by injecting an **`IHttpContextAccessor`** object.
  - e.g.:

```
public class SomeService {
  private readonly HttpContext _httpContext;
  public SomeService(IHttpContextAccessor
                     accessor) {
    _httpContext = accessor.HttpContext;
  }
}
```

AUTHENTICATION & AUTHORIZATION

Both web page controller and the application programming interface (API) of web services should be adequately protected

- *authentication*: it may be necessary to verify the identity of the user when accessing endpoints
- *authorization*: when accessing endpoints, it may be necessary to check that the user is authorized to perform the operation

Even the description of the API is not publicly available (e.g. through a public OpenAPI specification), endpoints can still be known

We cannot expect the user to use only the specified clients, the user can send any HTTP request at any time!

AUTHORIZATION

Access to resources can be restricted to logged in users

- the **`Authorize`** attribute can be applied to controllers or action methods, so that the resource can only be used after proper authorization (automatically enforced by the framework)
- e.g.:
```
[Authorize] // only for logged in users
public IActionResult ManageAccount(){ ... }
```
- access can be restricted to **`Users`** and **`Roles`**
- when authroization is applied on the controller level, certain action methods can be exempted with the **`AllowAnonymous`** attribute

AUTHORIZATION

- e.g.:

```
[Authorize] // applied to all action methods
public class AccountController : Controller
{
  public IActionResult ManageAccount(){ … }

  [Authorize(Roles = "admin")]
  // only users with the admin role can access this
  public IActionResult ManageAllAccounts() { … }

  [AllowAnonymous] // available for everyone
  public IActionResult Login(){ … }
  // ...
}
```

SECURE COMMUNICATION

Communicating on a secure channel, encrypting messages using *Transport Layer Security* (TLS, SSL) is essential in most cases

- configuring to use secure communication channels and enforcing them is the task of the webserver
- otherwise interception of communication (including *man in the middle* attacks), *session theft* attacks become trivial
- a webpage can expect a secure channel for a resource (with the **`RequireHttps`** attribute) or check for it with the **`Request.IsHttps`** attribute dynamically

SECURE COMMUNICATION

Possible solutions:

- Do not listen on unsecure channels at all, use only HTTPS
- Redirect all HTTP communication to HTTPS
  `app.UseHttpsRedirection();`
- Additionally, use HSTS as well to cache this setting at the clients
  `app.UseHsts();`
- Require HTTPS only for certain controller or action method
  (not advised in general, especially for WebAPI projects)

```
[Authorize]
[RequireHttps] // available only through secure channel
public class AccountController : Controller { … }
```

EXAMPLE

Task: Implement user management for the travel agency website in the ASP.NET WebAPI backend + Angular frontend architecture.

- Utilize *ASP.NET Core Identity*, the **`TravelAgencyContext`** type specializes the **`IdentityDbContext`** type. **`User`** type is a specialization of **`IdentityUser`**, extending it with full name and address.
- The framework provided **`UserManager`** and **`SignInManager`** services are used for user management.
- Extend the **`Program`** class with the proper configuration.
- When logged in, personal data is auto-filled for booking an apartment. Booking is still possible as a visitor, without account.

ACCESS TOKENS

Due to the lifecycle of web applications, the state is not preserved between two requests. Direct authentication of each request by the user (e.g. with username-password pair) is cumbersome

Access token based authentication can provide an automated solution

- after successful username-password authentication, an access token is provided to the client and persisted in the application
- for further requests, the client can send the previously received access token instead of the username-password pair
- secure, encrypted channel (HTTPS) is a critical element of security

ACCESS TOKENS

There are multiple approaches to pass access tokens between the client and the server

- in machine-to-machine communication (between applications)
  - in a GET/POST parameter, or
  - HTTP header (conventionally named **`Authorization`**)
- in machine-to-human communication (e.g. a human user via a browser) typically in a cookie
  - *ASP.NET Core Identity* uses the **`.AspNetCore.Identity.Application`** cookie for this by default
  - in the application, the required information is stored in the **`SecurityStamp`** field of the **`AspNetUsers`** table

ACCESS TOKENS

Typically, an access token is valid only for a specific period of time

- It can have a *sliding expiration*, i.e. it is extended for a limited period of time after each successful request
  - Typically used for cookie-based authentication
- Typically, a fixed lifetime access token is used for inter-application communication
  - When logging in, in addition to the *access token*, a *refresh token* is also provided to client and stored
  - New access tokens can be requested using the refresh token
  - The access token expires frequently (*e.g. 30 minutes*), the refresh token expires less frequently (*e.g. 30 days*) or not at all

ACCESS TOKENS

There are two types of access tokens:

- *opaque tokens*, where the token refers to a resource stored on the server, but does not carry any content on its own
- *transparent tokens*, where the token carries encrypted content, so that it can be used not only for authentication but also for authorization or to carry any information in general
    - the token can be digitally signed, so that it can be verified that its content has not been modified since it was issued by the server
    - the token can even be digitally encrypted if it is important that its content is not easily readable
    - such a specification is the *JSON Web Tokens* (JWT) standard

JSON WEB TOKENS (JWT)

JSON Web Tokens (JWT) are used to securely transfer information between applications

- After a successful username-password authentication, a server generates a JWT that contains other information (e.g. access privileges) in addition to the user's identity, called *claims*. The JWT is digitally signed by the server and delivered to the client.
- The client can read the JWT (if not encrypted) and return it to the server as an access token for further requests.
- The server checks that the content of the received JWT matches the digital signature and has not been *tampered* with. If it is OK, its content is authentic, therefor e.g. it is not necessary to load the privilege roles directly from the database.

JSON WEB TOKENS (JWT)

The JWT token is in JSON format and consists of three parts:

- *Header*: metadata about the token
- *Payload*: the content carried by the token
- *Signature*: the digital signature of the token

The three parts of the JWT token are encoded in base64 to avoid character encoding problems and are sent separated by dots (so they can be passed even as GET parameters)

- format: `header.payload.signature`, e.g.:
  eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJodHRwczovL2VsdGUuaHUvIiwic3ViIjoiMTIzNDU2Nzg5IiwibmFtZSI6IlRlc3p0IEVsZWsiLCJhZG1pbiI6dHJ1ZX0.U56N4FGI21Dw2GVAACzv7PRxTck7jvsszN23GKHn4ak

JSON WEB TOKENS (JWT)

- The JWT *header* specifies the used algorithm (HMAC SHA256, RSA, ECDSA) for digital signature and the type of the token (which is JWT).

```
{
  "alg": "HS256",
  "typ": "JWT“
}
```

- The JWT *payload* contains the carried information, e.g.:

```
{
  "iss": "https://elte.hu/",
  "sub": "123456789",
  "name": "Test User",
  "admin": true
}
```

JSON WEB TOKENS (JWT)

The payload contains the JWT claims, which fall into 3 categories:

- There are so-called *registered claims*, predefined in the JWT specification, such as **`iss`** (*issuer*), **`exp`** (*expiration time*) or **`sub`** (*subject*). Registered claims all have a 3-character name for brevity. They are essential to the purpose of JWT.
- There are also conventionally used *public claims* such as **`name`**, **`email`** or **`birthdate`**. They are widely used in many applications, so using the same claim names is beneficial.
- You can also define any private claim you like, such as **`admin`** in the previous example.

JSON WEB TOKENS (JWT)

The JWT *signature* is the digital signature of the *header* and *payload*, encoded in base64.

- When using the HMAC SHA256 algorithm, a *hash value* is generated using a *secret key* with the following algorithm (pseudo-code):

```
signature = HMACSHA256(
  base64(header) + "." + base64(payload),
  secret)
```

- The key is provided by the server (as a configuration value) and should be known only by the server issuing the digital signature.

CONFIGURE ASP.NET CORE FOR JWT

ASP.NET Core provides integrated support for JWT-based authentication and authorization

- at the **`Authorize`** attribute it can be specified to use JWT instead of the default identity token based authorization (**`JwtBearerDefaults.AuthenticationScheme`**)

As a more robust approach, on the **`WebApplicationBuilder`** object in the **`Program`** class, you can also configure this as the new default:

```
builder.Services.AddAuthentication(options => {
  options.DefaultAuthenticateScheme =
    JwtBearerDefaults.AuthenticationScheme;
  options.DefaultChallengeScheme =
    JwtBearerDefaults.AuthenticationScheme;
});
```

CONFIGURE ASP.NET CORE FOR JWT

- Here we can also enable the JWT-based authentication (**AddJwtBearer**) and configure the token validation:

```
builder.Services.AddAuthentication(...)
   .AddJwtBearer(options => {
     options.TokenValidationParameters =
       new TokenValidationParameters
       {
         ValidIssuer = "https://elte.hu", // validate issuer
         IssuerSigningKey =
           new SymmetricSecurityKey(
             Encoding.UTF8.GetBytes("secret key"))
         // validate digital signature
       };
});
```

CREATE JWT TOKENS

- Now, after a successful authentication with username-password pair, a JWT can be generated for the client:

```
var tokenDesc = new SecurityTokenDescriptor {
  Subject = new ClaimsIdentity(new[] {
              new Claim(ClaimTypes.Name, user.UserName)
  }), // claims added to the JWT
  Issuer = "https://elte.hu",
  Expires = DateTime.Now.AddMinutes(30),
  SigningCredentials = new SigningCredentials(
    new SymmetricSecurityKey(// configure digital signature
      Encoding.UTF8.GetBytes("secret key")),
      SecurityAlgorithms.HmacSha256)
};
```

CREATE JWT TOKENS

- The JWT may also contain authorization information, such as the user's roles, which could be added like:

```
var roles = await userManager.GetRolesAsync(user);
foreach (var role in roles)
  tokenDesc.Subject.AddClaim(
    new Claim(ClaimTypes.Role, role));
```

- Create the JWT string:

```
var handler = new JwtSecurityTokenHandler();
var jwtToken = handler.CreateToken(tokenDesc);
string tokenStr = handler.WriteToken(jwtToken);
```

- Finally, the JWT token can be returned to the client in the response body or even in an HTTP header:

```
Response.Headers.Add("Bearer", tokenStr);
```

PASSING JWT FROM SERVER TO CLIENT

- On the client side, the JWT token can be read from the HTTP message header of the response:

```
using (HttpClient client = new HttpClient())
{
  HttpResponseMessage response = await
    client.GetAsync("api/account/login");
  if (response.IsSuccessStatusCode)
  {
    token = response.Headers.GetValues("Bearer")
                            .FirstOrDefault(); // JWT token
  }
}
```

PASSING JWT FROM CLIENT TO SERVER

- For further requests, the client adds the JWT token to the *Authorization* header with the *Bearer* prefix (e.g. **`Bearer eyJhb...n4ak`**), where ASP.NET Core will look for it.

```
using (HttpClient client = new HttpClient())
{
  client.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", token);
  // add JWT token to Authorization HTTP header
  HttpResponseMessage response =
    await client.PostAsJsonAsync("…", <object>);
  // … await response
}
```

CLIENT APPLICATIONS

- Web services decouples backend and frontend, they allow the general usage of the server-side backend application in various environments.
- A client application of any platform can provide a client-side interface to a web service.
  - Can be a desktop application, a mobile application, a website
  - Can be used by other web services
- The backend web service and frontend clients can be developed in parallel on different platforms and programming languages by different development teams.

3RD PARTY CLIENT APPLICATIONS

- With JWT-based authentication and authorization, any client application can connect to our server-side service to perform operations by authorizing users
  - e.g. our Spotify account can post to our Facebook wall
- The authentication and authorization protocol is controlled by a separate specification (e.g. *OAuth2*, *OpenId*), JWT can be the format of the tokens (mandatory for *OpenId*, but not for *OAuth2*)
  - The protocol describes the type of tokens, their content, how they are issued and revoked, etc.
- Alternatives exist, e.g. SAML (*Security Assertion Markup Language*) defines both the protocol and its own XML-based token format.